



UNIVERSITAT_{DE}
BARCELONA

Grado en Ingeniería Informática

Trabajo de Final de Grado

Comparativa de Modelos de Aprendizaje Automático y Estrategias de Aumentación de Datos para la Detección de Fraude en Transacciones Financieras

Autor: Hlieb Sydorenko

Director: Bertran Miquel Oliver

Realizado en: Departamento de Matemáticas e Informática

Barcelona, 10 de junio de 2025

Agradecimientos

En primer lugar, quiero expresar mi más sincero agradecimiento a mi tutor de TFG, Bertran Miquel Oliver, por su orientación experta, paciencia y apoyo constante a lo largo de este proyecto.

Abstract

Financial fraud represents a critical challenge for the global economy, with annual losses exceeding \$28 billion worldwide. The exponential growth of digital transactions has created opportunities for sophisticated fraudulent activities that traditional rule-based systems struggle to detect effectively.

In this project, we develop and compare machine learning models for fraudulent transaction detection in financial environments, while evaluating data augmentation strategies to address extreme class imbalance. We use the “Credit Card Fraud Detection” dataset containing 284,807 transactions with only 0.17% fraud cases, implementing Random Forest, K-Nearest Neighbors, and Support Vector Machines with various kernel configurations.

Resumen

El fraude financiero representa un desafío crítico para la economía global, con pérdidas anuales que superan los 28.000 millones de dólares. El crecimiento exponencial de transacciones digitales ha creado oportunidades para actividades fraudulentas sofisticadas que los sistemas tradicionales basados en reglas no logran detectar eficazmente.

En este proyecto desarrollamos y comparamos modelos de aprendizaje automático para la detección de transacciones fraudulentas en entornos financieros, evaluando estrategias de aumentación de datos para abordar el desbalance extremo entre clases. Utilizamos el dataset “Credit Card Fraud Detection” con 284.807 transacciones y solo 0.17 % de casos fraudulentos, implementando Random Forest, K-Nearest Neighbors y Support Vector Machines con diversas configuraciones de kernel.

Resum

El frau financer representa un repte crític per a l'economia global, amb pèrdues anuals que superen els 28.000 milions de dòlars. El creixement exponencial de transaccions digitals ha creat oportunitats per a activitats fraudulentament sofisticades que els sistemes tradicionals basats en regles no aconsegueixen detectar eficaçment.

En aquest projecte desenvolupem i comparem models d'aprenentatge automàtic per a la detecció de transaccions fraudulentament en entorns financers, avaluant estratègies d'augmentació de dades per abordar el desequilibri extrem entre classes. Utilitzem el dataset "Credit Card Fraud Detection" amb 284.807 transaccions i només 0.17 % de casos fraudulents, implementant Random Forest, K-Nearest Neighbors i Support Vector Machines amb diverses configuracions de kernel.

Índice general

1	Introducción	1
1.1	Contexto	1
1.2	Problema	2
1.3	Motivación	3
1.4	Objetivo principal	4
1.5	Objetivos específicos	4
2	Métodos actuales	5
2.1	Evolución de los métodos de detección de fraude	5
2.2	Técnicas para el manejo del desbalance de clases	6
2.3	La irrupción del Deep Learning	7
2.4	Modelos híbridos: combinando lo mejor de dos mundos	8
3	Dataset	9
3.1	Origen y contexto	9
3.2	Estructura y características	9
3.3	Análisis de los datos	10
3.3.1	Análisis de Amount	12
3.3.2	Análisis Temporal	13
3.4	Análisis de correlación y variables destacables	15
3.5	Detección de outliers	17
4	Metodología	18
4.1	Preprocesamiento	18
4.1.1	Ingeniería de Características	18
4.1.2	Normalización	19
4.1.3	Técnicas de balanceo de clases	20
4.2	Entrenamiento y Evaluación de Modelos	21
4.2.1	Carga de datos preprocesados	21
4.2.2	Modelos implementados	21
4.2.3	Proceso de entrenamiento	22
4.2.4	Métricas de evaluación	22

4.2.5	Almacenamiento de resultados	23
5	Resultados	24
5.1	Comparación General de Métricas	24
5.2	Análisis de Métricas Clave para Detección de Fraude	24
5.2.1	Métricas Principales	24
5.2.2	Curvas ROC y PR	27
5.3	Matrices de Confusión	27
6	Discusión y Conclusiones	29
6.1	Análisis Comparativo de Estrategias de Balanceo	29
6.1.1	Dataset Original: Las Limitaciones del Desbalance Extremo	29
6.1.2	Métodos de Oversampling	30
6.2	Rendimiento de los Modelos de Clasificación	30
6.2.1	Random Forest	30
6.2.2	K-Nearest Neighbors	31
6.2.3	Support Vector Machines	31
6.3	Comparación con Trabajos Previos: Competitividad con Eficiencia .	32
	Bibliografía	33
	Anexo	35
6.4	Matrices de Confusión	35
6.5	Curvas ROC	37
6.6	Curvas Precision-Recall	40

Capítulo 1

Introducción

1.1. Contexto

El fraude financiero representa actualmente uno de los mayores desafíos para el sistema económico global. En un entorno donde las transacciones digitales crecen exponencialmente, tanto las instituciones financieras como los usuarios se enfrentan a amenazas cada vez más sofisticadas y difíciles de detectar. Las cifras son alarmantes: según el Nilson Report (2021) [14], las pérdidas anuales por fraude con tarjetas de crédito superan los 28.000 millones de dólares a nivel mundial, con una tendencia claramente ascendente.

La transformación digital ha revolucionado nuestra forma de interactuar con el dinero. Hoy realizamos compras con un simple clic, transferimos fondos instantáneamente y gestionamos nuestras finanzas desde dispositivos móviles. Esta comodidad, sin embargo, viene acompañada de nuevos riesgos: los estafadores han desarrollado métodos cada vez más sofisticados, adaptándose con sorprendente rapidez a cada nueva medida de seguridad implementada por las entidades financieras.

El impacto del fraude va mucho más allá de las pérdidas económicas directas. Las instituciones financieras se ven obligadas a destinar recursos significativos a sistemas de seguridad, gestión de reclamaciones y compensaciones a clientes afectados. Quizás más preocupante aún es la erosión de confianza que generan estos incidentes. Cuando un usuario sufre un fraude o experimenta un falso positivo que bloquea su transacción legítima, su confianza en el sistema financiero digital se deteriora, lo que puede frenar la adopción de innovaciones tecnológicas que, paradójicamente, podrían mejorar la seguridad general del sistema.

1.2. Problema

La detección de fraude financiero presenta desafíos técnicos y operativos significativos que dificultan su abordaje efectivo. El más prominente de estos retos es el extremo desbalance entre transacciones legítimas y fraudulentas en los conjuntos de datos disponibles. En un dataset típico de transacciones con tarjeta de crédito, menos del 0.2 % corresponden a fraudes, creando una asimetría que complica enormemente el desarrollo de sistemas de detección eficaces.

Este desbalance tiene consecuencias directas en el rendimiento de los modelos de clasificación. Los algoritmos tienden naturalmente a favorecer la clase mayoritaria, lo que en la práctica significa que pueden alcanzar una exactitud global aparentemente excelente (superior al 99 %) simplemente clasificando todas las transacciones como legítimas. Sin embargo, esta estrategia falla completamente en el objetivo principal: identificar las escasas pero costosas transacciones fraudulentas.

Además, las métricas tradicionales de evaluación resultan engañosas en estos contextos. La exactitud, por ejemplo, no refleja adecuadamente el rendimiento real del sistema cuando las clases están tan desproporcionadas. Un modelo que nunca detecte un solo fraude podría parecer exitoso según esta métrica, mientras que en realidad sería completamente inútil para su propósito, ya que el modelo podría estar clasificando todas las muestras en la clase mayoritaria.

La escasez de ejemplos de fraude también dificulta que los algoritmos aprendan patrones significativos. El aprendizaje automático se basa fundamentalmente en la disponibilidad de datos suficientes para identificar regularidades, y cuando una clase está tan pobremente representada, los modelos se centran en aprender las características de la mayoría del dataset, obviando hasta cierto punto esas clases minoritarias.

Por otra parte, los sistemas convencionales de detección, basados principalmente en reglas predefinidas, muestran limitaciones cada vez más evidentes en el entorno digital actual. Su rigidez adaptativa contrasta con la naturaleza evolutiva del fraude: mientras los estafadores estudian constantemente estos sistemas y desarrollan nuevas estrategias para eludirlos, las reglas estáticas quedan rápidamente obsoletas, creando un ciclo de actualización constante que resulta insostenible.

La escalabilidad representa otro obstáculo significativo. El comercio digital actual genera millones de transacciones por minuto, un volumen que los sistemas tradicionales, diseñados para entornos mucho menos intensivos, difícilmente pueden

procesar y analizar en tiempo real sin comprometer la exactitud o la velocidad de respuesta.

1.3. Motivación

La motivación principal de este trabajo surge de la necesidad de superar las limitaciones de los métodos tradicionales de detección de fraude mediante la aplicación de técnicas avanzadas de aprendizaje automático. ML ofrece un enfoque radicalmente distinto, con ventajas que lo hacen especialmente adecuado para este desafío.

A diferencia de las reglas estáticas, los modelos de ML poseen una capacidad de adaptación inherente, pudiendo actualizarse y evolucionar constantemente. Esta característica resulta invaluable en el contexto del fraude financiero, donde las estrategias fraudulentas cambian continuamente. Estos algoritmos pueden aprender nuevos patrones de fraude sin requerir una intervención manual constante.

Otra ventaja fundamental es su capacidad para detectar patrones complejos. Estos algoritmos pueden descubrir relaciones ocultas en los datos que serían imposibles de programar manualmente o descubrir mediante métodos estadísticos. Donde el ojo humano o las reglas predefinidas fallan, estos modelos pueden identificar señales distribuidas entre múltiples variables, reconociendo indicios de fraude que escaparían a métodos más tradicionales.

El aprendizaje automático también ofrece soluciones específicas para el problema del desbalance extremo entre clases. Técnicas como la generación de ejemplos sintéticos o los métodos de ponderación abordan directamente esta dificultad, permitiendo que los modelos aprendan efectivamente incluso cuando los ejemplos de fraude son escasos en comparación con las transacciones legítimas. Estas ponderaciones permiten que los modelos no solo aprenden patrones de las clases mayoritarias, sino también se centren en esas clases minoritarias que también son de interés.

La escalabilidad es otra ventaja importante. Los sistemas basados en ML pueden adaptarse con facilidad al crecimiento del comercio electrónico, gestionando grandes volúmenes de transacciones sin perder eficacia. Una vez el algoritmo ha sido entrenado en un conjunto de datos representativo, se pueden procesar millones de datos en tiempo real y de manera eficiente, sin necesidad de infraestructuras excesivamente complejas.

1.4. Objetivo principal

El objetivo principal de este proyecto es desarrollar y comparar diferentes modelos de ML para la detección de transacciones fraudulentas en entornos financieros, evaluando simultáneamente el impacto de diversas estrategias de aumentación de datos para abordar el problema del desbalance de clases inherente a este tipo de conjuntos de datos.

1.5. Objetivos específicos

1. **Análisis exploratorio del conjunto de datos:** Realizar un análisis exhaustivo del dataset “Credit Card Fraud Detection” [15], comprendiendo su estructura, distribución y características principales para fundamentar las decisiones metodológicas posteriores.
2. **Implementación de técnicas de preprocesamiento:** Aplicar métodos de normalización, tratamiento de valores atípicos, ingeniería de características y otras técnicas de preprocesamiento necesarias para optimizar el rendimiento de los algoritmos.
3. **Evaluación de estrategias de aumentación de datos:** Implementar y comparar diferentes técnicas para manejar el desbalance extremo entre clases, incluyendo SMOTE [2], ADASYN [6] y SMOTETomek [17], determinando su impacto en el rendimiento de los modelos.
4. **Desarrollo de modelos:** Implementar diversos algoritmos de clasificación (K-NN, SVM, Random Forest) bajo diferentes configuraciones.
5. **Establecimiento de un marco de evaluación robusto:** Seleccionar e implementar métricas apropiadas para evaluar correctamente problemas con clases desbalanceadas, como Precisión, Recall, F1-Score, AUC-ROC y AUC-PR, que permitan una evaluación justa y completa.
6. **Análisis e interpretación de resultados:** Realizar una comparativa de los resultados según la técnica de aumentación de datos y el modelo aplicados.

Capítulo 2

Métodos actuales

2.1. Evolución de los métodos de detección de fraude

La detección de fraude financiero ha experimentado una notable evolución en las últimas dos décadas, pasando de sistemas basados en reglas predefinidas a sofisticados algoritmos de aprendizaje automático. Este cambio de paradigma ha sido impulsado por la creciente complejidad de los patrones fraudulentos y el volumen masivo de transacciones que deben analizarse en tiempo real. Sin embargo, estos sistemas enfrentan el gran problema del desbalance de datos.

Dal Pozzolo et al. (2015) [1] marcaron un hito importante al establecer un marco comparativo para evaluar algoritmos en condiciones de desbalance extremo, una característica inherente a los conjuntos de datos de fraude financiero. Su investigación comparó sistemáticamente diversos algoritmos como Random Forest, Support Vector Machine (SVM), Regresión Logística y Naive Bayes. Quizás más importante aún, su trabajo puso de manifiesto la necesidad de utilizar métricas de evaluación específicas para contextos desbalanceados, como el área bajo la curva ROC (ROC-AUC) y las curvas de precisión-exhaustividad (Precision-Recall), en lugar de la exactitud global que puede resultar no representativa.

En paralelo, Whitrow et al. (2009) [4] exploraron un aspecto fundamental en la detección de fraude: el aprendizaje sensible al coste. Reconociendo que no todos los errores tienen el mismo impacto (un falso negativo puede costar miles de euros mientras que un falso positivo genera principalmente inconvenientes), desarrollaron variantes de Random Forest que incorporaban esta asimetría en su proceso de aprendizaje. Sus experimentos demostraron que los métodos de ensemble combinados con técnicas de muestreo balanceado superan a los modelos individuales, estableciendo

así una dirección clara para futuros desarrollos.

La ingeniería de características, otro pilar fundamental en la detección efectiva de fraude, fue abordada extensivamente por Bahnsen y colaboradores (2016) [3]. Su trabajo se centró en la creación de características derivadas basadas en el comportamiento histórico y patrones temporales de las transacciones. Esta aproximación resultó particularmente valiosa, logrando incrementar el rendimiento de los modelos en un 13 % según la métrica F1-score, evidenciando que la calidad de las características iniciales puede ser tan importante como la sofisticación del algoritmo utilizado o como las métricas para medir el rendimiento del algoritmo.

2.2. Técnicas para el manejo del desbalance de clases

El desbalance extremo entre transacciones legítimas y fraudulentas representa uno de los mayores desafíos técnicos en este campo. En un conjunto típico de transacciones financieras, menos del 0.2 % de estas corresponden a fraudes, lo que complica enormemente el entrenamiento efectivo de los modelos. Esta problemática ha motivado el desarrollo de técnicas específicas para equilibrar artificialmente los conjuntos de datos.

Chawla et al. (2002) [2], propuso Synthetic Minority Over-sampling Technique (SMOTE), un método que genera ejemplos sintéticos de la clase minoritaria interpolando entre instancias reales. Esta técnica, que ha llegado a convertirse en un estándar, demostró mejoras significativas en la detección de fraudes cuando se aplica antes del entrenamiento del modelo, permitiendo que los algoritmos capturen tanto los patrones subyacentes de las transacciones fraudulentas como los de las transacciones legales.

He et al. (2008) [6] crearon Adaptive Synthetic Sampling (ADASYN), una mejora del método SMOTE que funciona de manera más inteligente. En lugar de generar la misma cantidad de datos sintéticos en todas partes, ADASYN se fija en qué casos de fraude son más difíciles de detectar para el algoritmo. Donde los casos de fraude se parecen mucho a las transacciones normales, genera más ejemplos sintéticos para ayudar al modelo a aprender mejor esos patrones confusos. Es como si un profesor dedicara más tiempo a explicar los conceptos que resultan más difíciles para sus estudiantes.

Estas técnicas que se utilizan para ampliar la cantidad de datos tienen el nombre de Oversampling.

Por otra parte, Liu et al. (2009) [7] abordaron el problema desde la perspectiva opuesta con EasyEnsemble, una técnica de undersampling que combina múltiples submuestras aleatorias de la clase mayoritaria. Este método permite utilizar toda la información disponible de la clase minoritaria mientras reduce la cantidad de datos de la clase mayoritaria, así manteniendo un balance adecuado durante el entrenamiento.

Más recientemente, las técnicas híbridas como SMOTETomek [17] han ganado popularidad al combinar oversampling y undersampling en un solo proceso, eliminando ejemplos de la clase mayoritaria que podrían generar ruido después de la generación de instancias sintéticas.

2.3. La irrupción del Deep Learning

Aunque los métodos tradicionales de ML han dominado históricamente el campo de la detección de fraude, Deep Learning (DL) ha comenzado a ganar terreno en los últimos años, especialmente para capturar patrones temporales complejos y relaciones no lineales entre variables.

Jurgovsky et al. (2018) [5] aplicaron dos tipos de redes, Long Short-Term Memory (LSTM) y Recurrent Neural Networks (RNN), para modelar secuencias de transacciones. Su investigación reveló que estos modelos son capaces de capturar patrones temporales que escapan a los métodos tradicionales.

Un avance significativo en la aplicación de Deep Learning para la detección de fraudes fue desarrollado por Pang, Shen y van den Hengel (2019) [8] con las Deviation Networks. A diferencia de métodos anteriores, este enfoque aprende directamente a asignar una puntuación numérica que indica qué tan sospechosa o anómala es cada transacción, donde valores más altos representan mayor probabilidad de fraude. Su principal ventaja es la capacidad de funcionar eficientemente con pequeñas cantidades de anomalías etiquetadas.

2.4. Modelos híbridos: combinando lo mejor de dos mundos

Una tendencia emergente y prometedora es el desarrollo de arquitecturas híbridas que combinan las fortalezas de los modelos basados en árboles con las capacidades de las redes neuronales

Liu et al. (2012) [11] desarrollaron Isolation Forest, un método no supervisado que aísla observaciones anómalas mediante particiones aleatorias del espacio de características. Su principio fundamental es que las anomalías requieren menos particiones para ser aisladas, lo que proporciona una medida natural de rareza o anomalía.

Otros enfoques avanzados incluyen el uso de autoencoders para detección de anomalías, como propusieron Paula et al. (2016) [10] en su trabajo sobre investigación de fraude en exportaciones brasileñas y lavado de dinero. Estos modelos aprenden a reconstruir transacciones normales y utilizan el error de reconstrucción como indicador de anomalía.

En esta línea, Sarkar (2021) [9] desarrolló XBNet (Extremely Boosted Neural Network), una arquitectura que combina de manera inteligente dos enfoques: los árboles de decisión potenciados y las redes neuronales convencionales. El funcionamiento de XBNet se basa en un proceso de dos fases. Inicialmente, utiliza XGBoost para determinar cuáles son las características más relevantes en los datos y configura los pesos de la red neuronal priorizando aquellos patrones que aparecen con menor frecuencia. Después, aplica el entrenamiento neuronal tradicional mediante Gradient Descent para refinar estos pesos, logrando un equilibrio entre la atención especial a los casos minoritarios y el rendimiento predictivo general del sistema.

Capítulo 3

Dataset

3.1. Origen y contexto

El dataset “Credit Card Fraud Detection” [15] utilizado en este trabajo proviene de una colaboración entre investigadores de la Universidad Libre de Bruselas (ULB) y una institución financiera europea que no ha sido revelada públicamente por razones de confidencialidad. Estos datos, dada a su disponibilidad a través de la plataforma Kaggle, se han adoptado como referencia para evaluar algoritmos de detección de fraudes en transacciones financieras.

Los datos corresponden a transacciones realizadas con tarjetas de crédito por titulares europeos durante un período de dos días en septiembre de 2013. Esta temporalidad limitada pero intensiva permite analizar patrones de comportamiento en un marco temporal coherente, minimizando la influencia de factores estacionales o cambios en las tendencias de consumo que podrían afectar a conjuntos de datos más extendidos en el tiempo.

3.2. Estructura y características

El dataset está compuesto por un total de 284,807 transacciones, de las cuales únicamente 492 están etiquetadas como fraudulentas. Esta distribución refleja fielmente uno de los mayores desafíos en la detección de fraude financiero: el extremo desbalance entre clases, con apenas un 0.172 % de transacciones fraudulentas. Esta proporción, aunque complica significativamente el proceso de modelado, representa con precisión la realidad del problema en un entorno real, donde los fraudes constituyen una fracción mínima del volumen total de operaciones.

En cuanto a su estructura, el dataset contiene 31 variables:

- **Time:** Representa el tiempo transcurrido en segundos entre cada transacción y la primera transacción del conjunto de datos. Esta variable permite analizar la distribución temporal de las transacciones y potencialmente identificar patrones relacionados con momentos específicos del día.
- **Amount:** Corresponde al valor monetario de cada transacción. Esta variable no ha sido transformada y se presenta en su unidad original (euros), permitiendo analizar si existe relación entre el monto de la operación y la probabilidad de fraude.
- **V1 - V28:** Constituyen 28 características anónimas que resultan de aplicar Análisis de Componentes Principales (PCA) a las variables originales. Esta transformación responde a la necesidad de confidencialidad al ser un entorno bancario, pero también aporta beneficios técnicos al reducir la dimensionalidad y eliminar posibles correlaciones entre variables. Cada una de estas componentes captura diferentes aspectos del comportamiento transaccional, aunque su interpretación directa no es posible debido a la naturaleza de la transformación PCA. La aplicación de PCA como método de anonimización representa tanto una ventaja como un desafío. Por un lado, proporciona variables ya preprocesadas que capturan la variabilidad de los datos originales de manera eficiente. Por otro lado, dificulta la interpretabilidad de los resultados, ya que no es posible asociar directamente estas componentes con características específicas del comportamiento del titular o de la transacción, como podrían ser ubicación geográfica, tipo de comercio o historial de compras.
- **Class:** Variable categórica binaria que toma el valor 1 para transacciones fraudulentas y 0 para transacciones legítimas. Esta es la variable objetivo que los modelos intentarán predecir.

3.3. Análisis de los datos

Para comenzar el análisis, cargamos el dataset desde el archivo `creditcard.csv` [15] en el entorno de programación Jupyter Notebook, herramienta que utilizaremos a lo largo del proyecto para análisis y visualización. Como ya establecimos anteriormente, nuestro dataset original contiene 284,807 filas cada una con 31 atributos.

Aunque anticipamos que el dataset contaría con datos limpios, realizamos una verificación para confirmar la ausencia de valores nulos. Esta comprobación confirma que no es necesario aplicar técnicas de limpieza o imputación de datos, lo cual

facilita considerablemente el proceso de análisis y evita la introducción de sesgos adicionales.

Continuamos nuestro análisis con la distribución de los datos, obteniendo la siguiente información:

- Total de transacciones: 284,807
- Transacciones normales: 284,315
- Transacciones fraudulentas: 492
- Porcentaje de fraudes: 0.1727 %
- Ratio de desbalance: 1:577.88

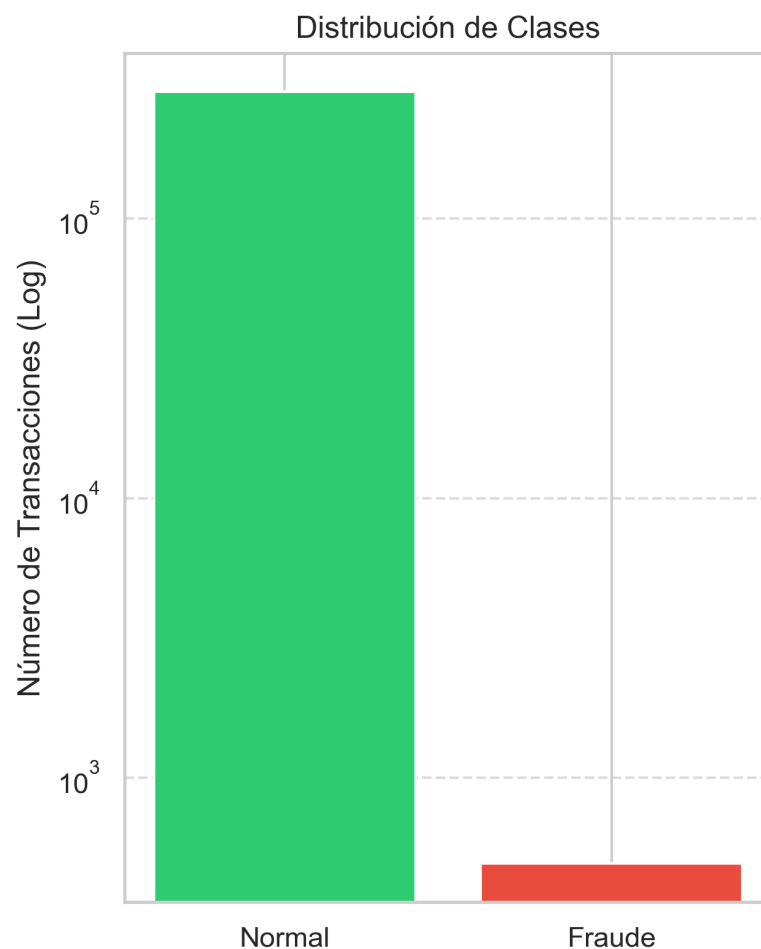


Figura 1: Distribución de clases en el dataset. Se observa el extremo desbalance entre transacciones normales (284,315) y fraudulentas (492), representando un ratio de aproximadamente 577:1. La escala logarítmica permite visualizar ambas clases a pesar de la gran diferencia en magnitud.

Estos números confirman el principal desafío del dataset: el extremo desbalance entre clases. Por cada transacción fraudulenta existen aproximadamente 578 transacciones legítimas.

Es importante destacar que este desbalance no representa una limitación del dataset, sino que refleja la realidad del fraude financiero. No obstante, confirma la necesidad de implementar técnicas específicas de muestreo para evitar que los modelos favorezcan a la clase mayoritaria.

3.3.1. Análisis de Amount

La variable 'Amount' representa el valor monetario de cada transacción y, a diferencia de las variables V1-V28, no ha sido anonimizada ni transformada mediante PCA. Esto nos permite analizarla directamente e interpretar sus valores en términos económicos reales.

El análisis estadístico revela diferencias interesantes entre las transacciones legítimas y fraudulentas. Las transacciones normales presentan un valor medio de 88.29€, mientras que las fraudulentas muestran un promedio ligeramente superior de 122.21€. Sin embargo, el rango de valores difiere significativamente: las transacciones legítimas alcanzan un máximo de 25,691.16€, mientras que las fraudulentas no superan los 2,125.87€.

Para visualizar estas diferencias, generamos tres boxplots complementarios: uno completo que muestra todo el rango de valores, otro representando el percentil 95 para apreciar mejor la distribución central, y un tercero con escala logarítmica para equilibrar la visualización de valores extremos.

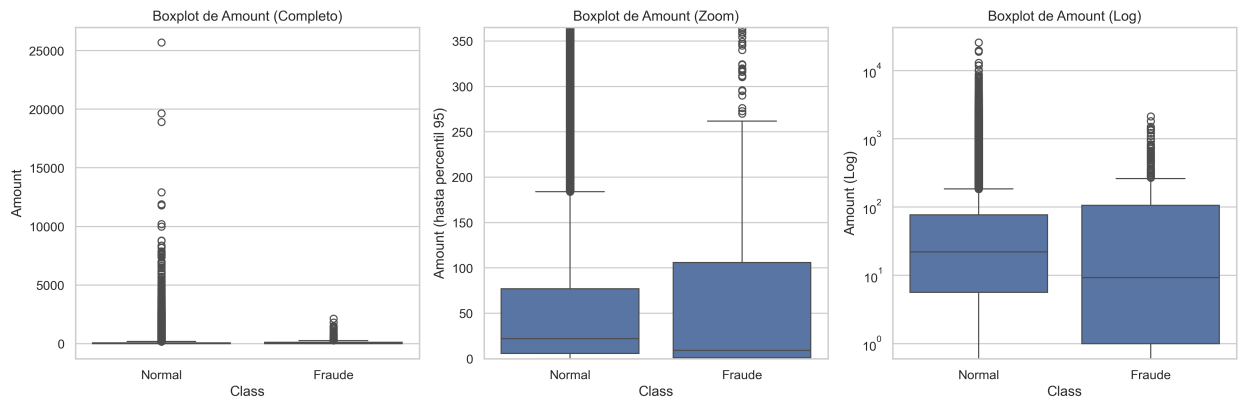


Figura 2: Análisis de la variable Amount mediante boxplots. (Izquierda) Vista completa mostrando el rango total de valores. (Centro) Zoom al percentil 95 para mejor visualización de la distribución central. (Derecha) Escala logarítmica para equilibrar la visualización de valores extremos. Las transacciones fraudulentas muestran un rango menor pero con mayor concentración de outliers.

3.3.2. Análisis Temporal

El análisis de la variable 'Time' proporciona información valiosa sobre los patrones temporales de las transacciones. Esta variable representa el tiempo transcurrido en segundos desde la primera transacción del dataset, abarcando un período de 48 horas (dos días consecutivos).

Para facilitar el análisis, se realizó una transformación de la variable 'Time' a formato horario, creando nuevas variables derivadas:

- **Time_Hour:** Conversión directa de segundos a horas (0 a 48 horas)
- **Hour_of_Day:** Hora del día en formato decimal (0 a 24 horas)
- **Hour_of_Day_Int:** Hora del día en formato entero (0 a 23) para agrupaciones

Aunque este aspecto será tratado en profundidad más adelante, esta transformación temporal constituye un ejemplo de cómo las variables originales pueden procesarse para extraer información más útil para el análisis y posterior modelado.

El análisis de la distribución porcentual de transacciones por hora del día revela patrones marcadamente diferentes entre transacciones legítimas y fraudulentas.

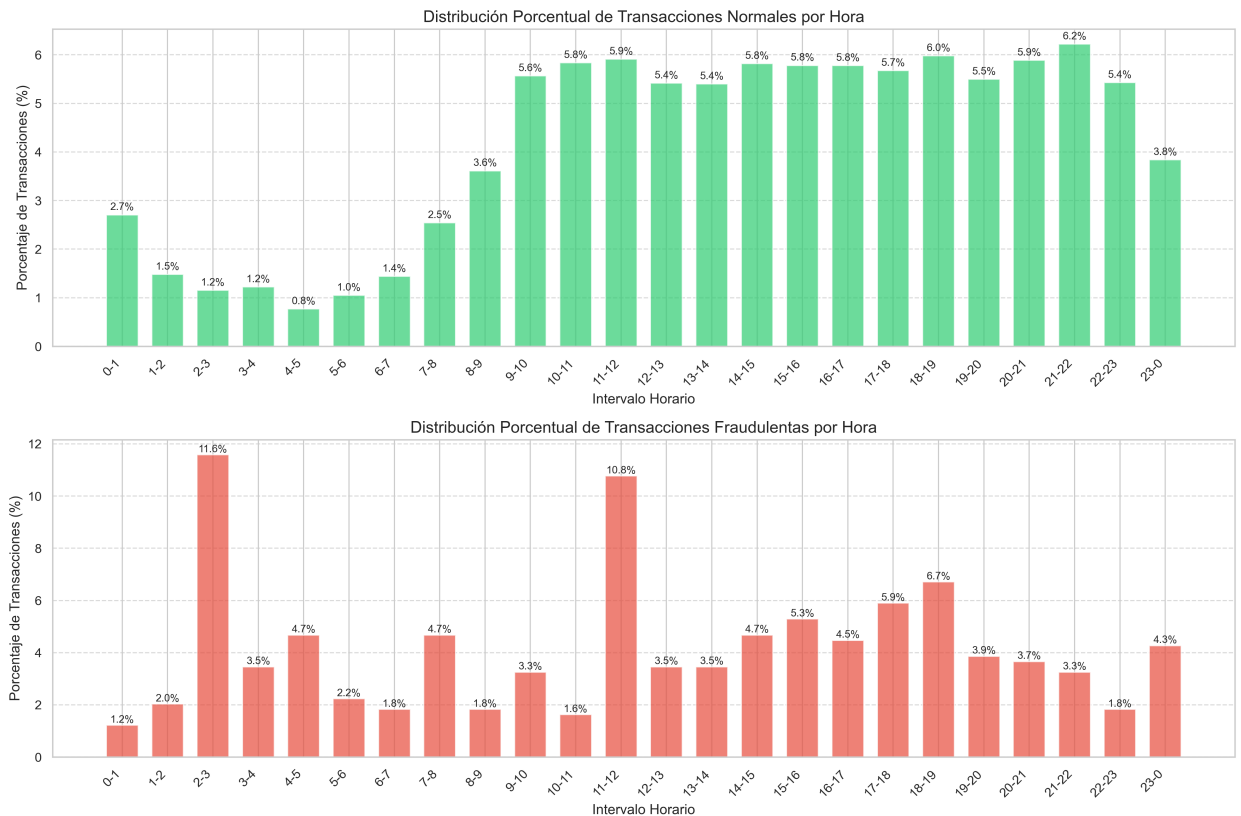


Figura 3: Distribución temporal separada por tipo de transacción. (Superior) Transacciones normales con actividad mínima durante madrugada (0.8-2.7 %) y máxima durante horas comerciales (5.4-6.2 %). (Inferior) Transacciones fraudulentas con distribución irregular y concentración en períodos específicos, evidenciando patrones de comportamiento anómalos.

Las transacciones legítimas muestran una distribución relativamente uniforme durante las horas de actividad comercial típica (9:00-22:00), con porcentajes que oscilan entre 5.4 % y 6.2 %. La actividad es mínima durante las horas nocturnas y de madrugada (0:00-8:00), con valores entre 0.8 % y 2.7 %, lo que refleja los patrones naturales de comportamiento humano y comercial.

Las transacciones fraudulentas presentan una distribución considerablemente más irregular, con tres picos de actividad claramente identificables:

- Pico de madrugada (2:00 - 3:00): 11.6 % de toda la actividad fraudulenta
- Pico de mediodía (11:00 - 12:00): 10.8 % de la actividad fraudulenta

Estos dos picos contrastan drásticamente con la distribución uniforme de las transacciones legítimas y representan más del 22 % de toda la actividad fraudulenta en apenas dos horas del día.

Este análisis temporal establece que la hora de la transacción constituye una característica discriminante valiosa para la detección de fraude. La clara asimetría en los patrones temporales entre clases proporciona información complementaria que puede mejorar significativamente el rendimiento de los modelos de clasificación.

3.4. Análisis de correlación y variables destacables

Para identificar las variables con mayor capacidad discriminante, se ha analizado la correlación de todas las características con la variable objetivo (Class). Este análisis permite priorizar las variables más relevantes para la detección de fraude y comprender mejor las relaciones lineales presentes en los datos.

El cálculo de correlaciones revela que ninguna variable individual presenta una correlación extremadamente alta con la clase objetivo, lo que indica que la detección de fraude es un fenómeno complejo que requiere la combinación de múltiples características.

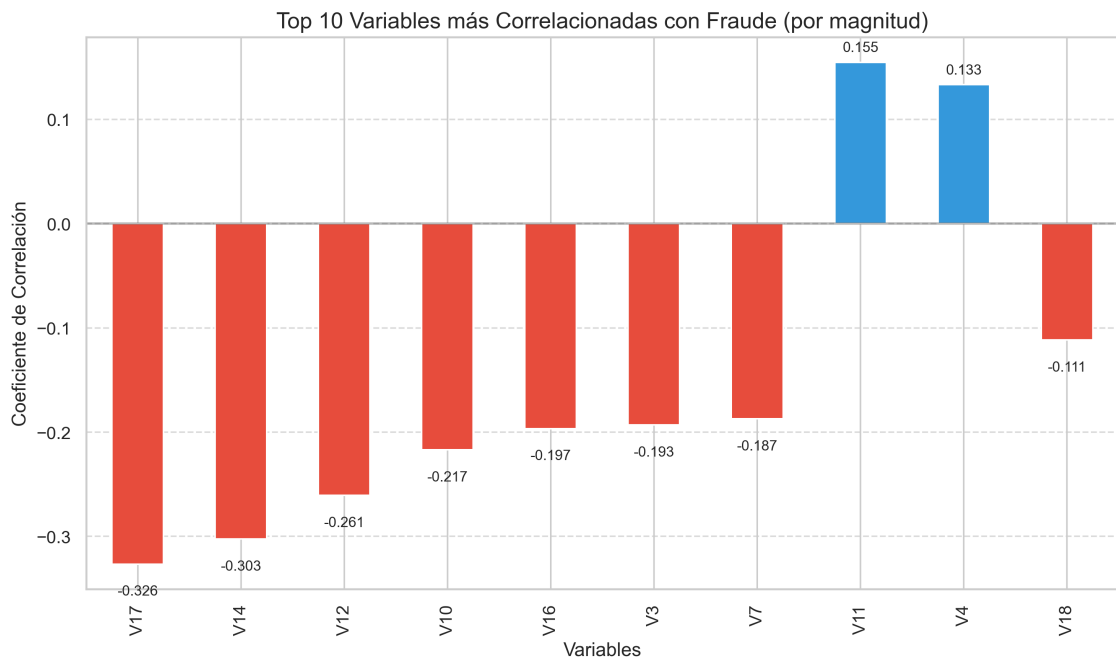


Figura 4: Top 10 variables más correlacionadas con la detección de fraude. V17 presenta la mayor correlación negativa (-0.326), seguida por V14 (-0.303) y V12 (-0.261). Las variables con correlación positiva como V11 (0.155) y V4 (0.133) también muestran capacidad discriminante. La mayoría de correlaciones son negativas, indicando que las transacciones fraudulentas tienden a concentrarse en valores menores de estas componentes principales.

La visualización de estas correlaciones muestra claramente que V17 destaca como la variable individual más correlacionada con la detección de fraude, seguida por V14 y V12. Es notable que la mayoría de las variables más discriminantes presentan correlaciones negativas, sugiriendo que las transacciones fraudulentas tienden a concentrarse en valores menores de estas componentes principales. Aun así hay variables correlacionadas positivamente como la V11 y la V4, implicando que el aumento de estas variables implica el aumento de la probabilidad de fraude.

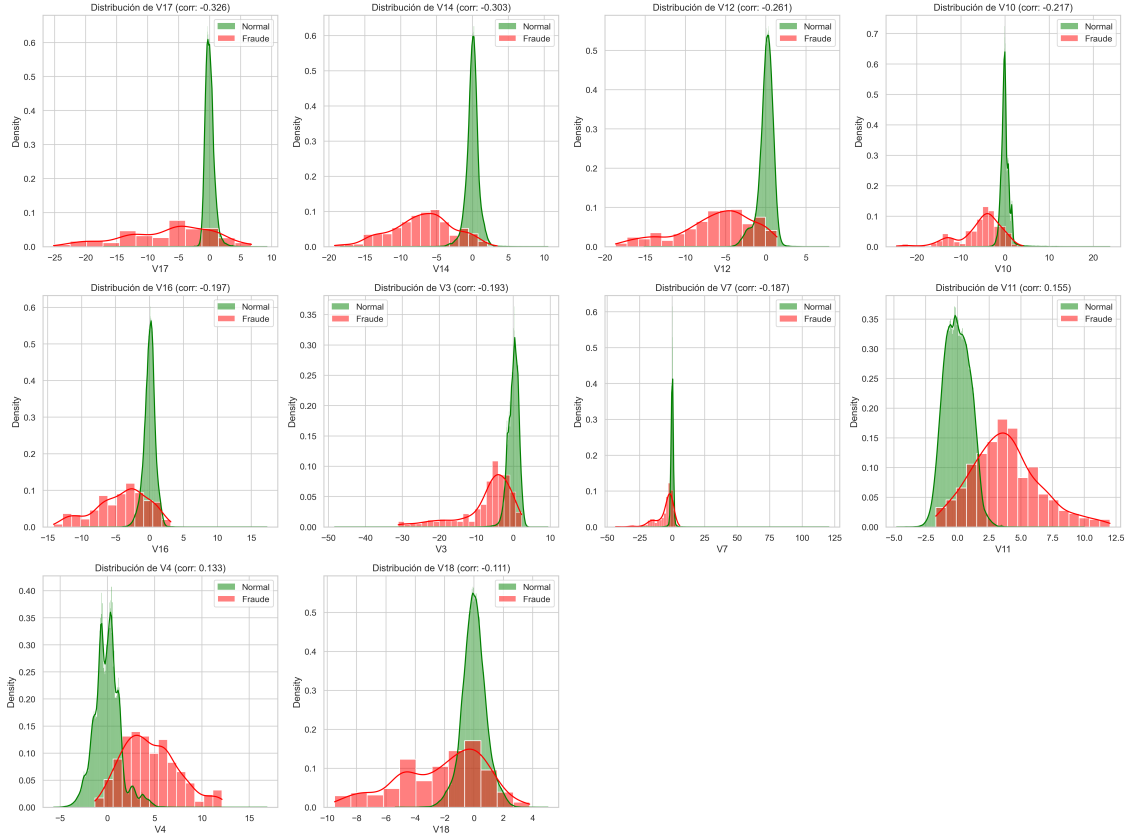


Figura 5: Distribuciones de las 10 variables más correlacionadas con la detección de fraude. Los histogramas superpuestos muestran clara separabilidad entre transacciones normales (verde) y fraudulentas (rojo). Las variables con correlación negativa (V17, V14, V12) presentan transacciones fraudulentas concentradas en valores negativos, mientras que las variables con correlación positiva (V11, V4) muestran el patrón inverso. Esta separación visual confirma el potencial discriminante de estas características para la clasificación.

El análisis de las distribuciones de las diez variables más correlacionadas proporciona información valiosa sobre la separabilidad entre clases. Los histogramas superpuestos revelan patrones distintivos:

Variables con alta correlación negativa: Muestran una clara separación entre clases, donde las transacciones normales se distribuyen principalmente alrededor

de cero, mientras que las fraudulentas se concentran en valores negativos. Esta separación visual confirma el potencial discriminante de estas variables.

Variables con correlación positiva: Presentan el patrón inverso, con las transacciones fraudulentas desplazadas hacia valores positivos. Aunque la correlación es menor en magnitud, la separación visual entre distribuciones sigue siendo evidente.

3.5. Detección de outliers

La presencia de valores atípicos puede afectar significativamente el rendimiento de muchos algoritmos de aprendizaje automático. Es importante destacar que este análisis de outliers se centra exclusivamente en la variable Amount, ya que las características anonimizadas (V1-V28) ya han sido normalizadas mediante PCA en el proceso de creación del dataset, lo que minimiza el impacto de posibles valores atípicos en estas variables.

Para identificar estos outliers en la variable Amount, aplicamos el método del Rango Intercuartílico (IQR), analizando por separado las transacciones normales y fraudulentas. Las transacciones normales presentan un 11.21 % de valores atípicos (31,862 casos), mientras que las transacciones fraudulentas muestran un 14.02 % de outliers (69 casos).

Esta mayor proporción de valores atípicos en transacciones fraudulentas sugiere que los comportamientos inusuales en términos de cantidad podrían ser indicativos de actividad sospechosa. Sin embargo, también plantea desafíos para el modelado, ya que estos valores extremos podrían distorsionar el aprendizaje si no se manejan adecuadamente.

La presencia de outliers en la variable Amount justifica la aplicación de técnicas de normalización robustas como RobustScaler, que son menos sensibles a valores extremos que otras técnicas como StandardScaler. Alternativamente, podríamos considerar transformaciones logarítmicas para la variable Amount, lo que reduciría el impacto de los valores extremos mientras preserva la información sobre el orden de magnitud de las transacciones.

Capítulo 4

Metodología

4.1. Preprocesamiento

4.1.1. Ingeniería de Características

Basándose en los patrones temporales identificados en el análisis exploratorio, se desarrollaron múltiples características derivadas de la variable 'Time':

- **Time_Hour:** Conversión directa de segundos a horas (0-48h)
- **Hour_of_Day:** Hora del día en formato decimal (0-24h)
- **Hour_of_Day_Int:** Hora del día como entero para agrupaciones (0-23)

Para capturar la naturaleza cíclica del tiempo, se implementaron transformaciones trigonométricas que preservan la continuidad temporal:

- **Hour_sin:** $\sin(2\pi \times \text{Hour_of_Day}/24)$
- **Hour_cos:** $\cos(2\pi \times \text{Hour_of_Day}/24)$

Estas transformaciones trigonométricas son especialmente importantes porque evitan que los algoritmos interpreten la hora 23 y la hora 0 como valores muy distantes, cuando en realidad son consecutivas. Esta representación cíclica es fundamental dado que el análisis exploratorio reveló patrones específicos de fraude durante la madrugada.

La variable 'Amount' mostró distribuciones altamente sesgadas con numerosos outliers. Para abordar esta característica se implementó:

Amount_Log: Transformación logarítmica que:

- Reduce el impacto de valores extremos
- Normaliza la distribución sesgada
- Preserva el orden relativo de los valores
- Mantiene la interpretabilidad económica

La selección de estas transformaciones se fundamenta tanto en los descubrimientos del análisis exploratorio como en las mejores prácticas documentadas en la literatura. Bahnsen et al. (2016) [3] demostraron que las características temporales derivadas pueden incrementar el rendimiento de los modelos hasta un 13 % en términos de F1-score.

Es importante destacar que las variables V1-V28, al ser resultado de una transformación PCA, ya se encuentran normalizadas y centradas. Por tanto, no requieren transformaciones adicionales y se mantienen en su forma original para preservar la información que capturan del comportamiento transaccional subyacente.

4.1.2. Normalización

La normalización de características constituye un paso crítico en el preprocesamiento, especialmente cuando se combinan variables con escalas muy diferentes y se utilizan algoritmos sensibles a la magnitud de los valores, como SVM y K-NN.

Dado que el dataset contiene variables con características muy distintas, se implementó una estrategia de normalización selectiva. Las componentes principales V1-V28 se mantuvieron sin modificar, ya que la transformación PCA produce automáticamente variables centradas en cero con varianza unitaria. Aplicar normalización adicional podría eliminar información valiosa contenida en estas componentes.

Se aplicó normalización únicamente a las variables 'Time' y 'Amount', que conservan sus escalas originales y presentan rangos muy diferentes: Time abarca de 0 a 172,800 segundos (48 horas), mientras que Amount varía de 0 a 25,691.16 euros.

Se eligió RobustScaler sobre otros métodos de normalización por su resistencia a outliers. El análisis exploratorio reveló que ambas clases presentan abundantes valores atípicos en 'Amount' (11.21 % en transacciones normales, 14.02 % en fraudulentas). RobustScaler utiliza la mediana y el rango intercuartílico en lugar de la media y

desviación estándar, siendo menos sensible a estos outliers. Al usar percentiles (25 % y 75 %) en lugar de valores extremos, este método mantiene la estructura relativa de los datos sin comprometer la información discriminante que pueden contener los valores atípicos. Además, en sistemas de detección de fraude en tiempo real pueden aparecer transacciones con cantidades inusuales, y RobustScaler proporciona mayor estabilidad ante estas variaciones.

La normalización se implementó siguiendo buenas prácticas para evitar filtración de datos. El proceso crea copias de los conjuntos originales para preservar los datos sin modificar, luego inicializa un RobustScaler que se ajusta exclusivamente con los datos de entrenamiento mediante `fit_transform` para las columnas 'Time' y 'Amount'. Posteriormente, se aplica la misma transformación al conjunto de prueba usando únicamente el método `transform`, garantizando que la información del conjunto de prueba no influya en el proceso de normalización.

4.1.3. Técnicas de balanceo de clases

El extremo desbalance entre clases identificado en el análisis exploratorio (0.1727 % de fraudes) representa el principal obstáculo para entrenar modelos efectivos. Los algoritmos de machine learning tienden naturalmente a favorecer la clase mayoritaria, lo que resulta en modelos que clasifican prácticamente todas las transacciones como legítimas. Para abordar esta problemática, se evaluaron tres técnicas de muestreo reconocidas en la literatura.

Las tres técnicas que aplicamos son métodos de oversampling explicados anteriormente: SMOTE, que crea ejemplos sintéticos de fraude mediante interpolación entre casos existentes; ADASYN, que se enfoca en generar más datos en las zonas más difíciles de clasificar; y SMOTETomek, que combina la generación sintética con una limpieza posterior del dataset para eliminar casos ambiguos.

La aplicación de estas técnicas transformó radicalmente la composición del conjunto de entrenamiento. Mientras que el dataset original mantenía su desbalance natural extremo, las 3 técnicas de Oversampling produjeron conjuntos perfectamente equilibrados con aproximadamente el mismo número de ejemplos de ambas clases.

Reconociendo que el proceso completo de preprocesamiento requiere tiempo computacional considerable, se diseñó una estrategia de almacenamiento que evitará repetir estos cálculos en experimentos futuros. Cada variante del dataset se guarda en formato .pt (PyTorch), que ofrece ventajas prácticas importantes.

4.2. Entrenamiento y Evaluación de Modelos

Una vez completado el preprocesamiento de datos y aplicadas las técnicas de balanceo, el siguiente paso crucial es el entrenamiento y evaluación de los modelos. Esta fase determina qué combinación de algoritmo y estrategia de balanceo ofrece el mejor rendimiento para la detección de fraude.

4.2.1. Carga de datos preprocesados

El proceso comienza con la carga de los conjuntos de datos previamente preprocesados y balanceados. Cada conjunto se almacenó en formato .pt (PyTorch), una elección que no solo preserva la precisión numérica sino que también permite cargas eficientes. Estos archivos contienen tanto el conjunto de entrenamiento específico para cada técnica de balanceo como el conjunto de prueba común para todas las variantes, además de los nombres de las características para facilitar la interpretación posterior.

4.2.2. Modelos implementados

Se implementaron cinco modelos diferentes, cada uno representando un enfoque distinto para abordar el problema:

Random Forest (RF) fue seleccionado como representante de los métodos de ensemble. Siguiendo las recomendaciones de Dal Pozzolo et al. (2015) [1], se configuró con 100 estimadores sin restricción de profundidad máxima, permitiendo que los árboles capturen patrones complejos en los datos. Este algoritmo es especialmente robusto ante outliers y puede manejar eficazmente datasets desbalanceados.

K-Nearest Neighbors (K-NN) representa los métodos basados en similitud. Se configuró con $k=5$ vecinos y ponderación por distancia, siguiendo las recomendaciones de Whitrow et al. (2009) [4] para detección de fraude. La ponderación por distancia permite que ejemplos más cercanos tengan mayor influencia en la decisión, lo que resulta especialmente útil en datasets donde los casos de fraude pueden formar clusters específicos.

Support Vector Machine (SVM) se implementó con tres variantes de kernel diferentes para evaluar su capacidad de modelar relaciones no lineales. El kernel

RBF (Radial Basis Function) es efectivo para capturar patrones complejos, el kernel polinómico de grado 2 modela relaciones cuadráticas, y el kernel sigmoide puede aproximar redes neuronales simples. Todos se configuraron con probabilidades habilitadas para calcular métricas como ROC-AUC.

4.2.3. Proceso de entrenamiento

El entrenamiento se realizó de manera sistemática y reproducible, aplicando cada modelo a los cuatro conjuntos de datos: original (desbalanceado), SMOTE, ADASYN y SMOTETomek. Este diseño experimental permite evaluar no solo el rendimiento individual de cada algoritmo, sino también cómo interactúa con las diferentes estrategias de balanceo. Para garantizar una comparación justa, los hiperparámetros de cada modelo se mantuvieron constantes a través de todos los conjuntos de datos, y todo el proceso se ejecutó con una semilla aleatoria fija (42) para asegurar la reproducibilidad de los resultados.

Durante el entrenamiento, registramos el tiempo requerido por cada combinación de modelo y conjunto de datos, proporcionando información valiosa sobre la eficiencia computacional.

4.2.4. Métricas de evaluación

La evaluación de los modelos se realizó mediante un conjunto completo de métricas, seleccionadas específicamente para el contexto de detección de fraude:

- **Accuracy:** Proporción de predicciones correctas
- **Precisión:** Proporción de alertas de fraude que son realmente fraudes
- **Recall:** Proporción de fraudes reales que son detectados
- **F1-Score:** Media armónica entre precisión y recall
- **ROC-AUC:** Área bajo la curva ROC, que evalúa el rendimiento a diferentes umbrales
- **PR-AUC:** Área bajo la curva Precision-Recall, especialmente relevante para clases minoritarias

- **False Positive Rate:** Proporción de transacciones legítimas incorrectamente clasificadas como fraude
- **Tiempo de entrenamiento**
- **Tiempo de predicción**

Además, se generó la matriz de confusión completa para cada modelo, permitiendo un análisis detallado de los verdaderos positivos (TP), falsos positivos (FP), verdaderos negativos (TN) y falsos negativos (FN).

4.2.5. Almacenamiento de resultados

Pensando en la utilidad práctica de los experimentos y en facilitar análisis futuros, se desarrolló una estrategia de almacenamiento que captura toda la información relevante de manera organizada. Cada modelo entrenado produce dos tipos de archivos que sirven propósitos diferentes pero complementarios.

Los modelos entrenados se guardan en formato pickle (.pkl), lo que permite cargarlos y utilizarlos directamente en el futuro sin necesidad de repetir el proceso de entrenamiento. Este formato de serialización es especialmente eficiente para objetos de Python y mantiene todos los parámetros y configuraciones del modelo intactos. Por separado, también en formato pickle, se almacenan todos los detalles de las predicciones realizadas durante la evaluación: las clasificaciones finales, las probabilidades que el modelo asignó a cada predicción, las etiquetas reales para comparación, y las matrices de confusión completas. Esta información es fundamental para realizar análisis profundos sobre cómo y cuándo fallan los modelos, identificar patrones en los errores cometidos, y generar las visualizaciones que ayuden a entender el comportamiento de cada algoritmo.

Además de estos archivos detallados, generamos tablas resumen en formato CSV para cada técnica de balanceado. Estas tablas concentran las métricas principales de todos los modelos evaluados bajo cada método, facilitando las comparaciones rápidas entre algoritmos.

Capítulo 5

Resultados

En esta sección presentamos los resultados obtenidos al evaluar los diferentes modelos de aprendizaje automático y estrategias de balanceo de datos. Los resultados se muestran de manera objetiva, sin entrar en discusión detallada, que se realizará en la siguiente sección. Hemos evaluado el rendimiento considerando un conjunto de métricas clave que nos permiten entender completamente cómo funcionan los modelos en la detección de fraudes en transacciones financieras.

5.1. Comparación General de Métricas

La Tabla 1 resume las métricas de rendimiento principales para cada modelo y técnica de Oversampling. Estas métricas incluyen Accuracy, Precision, Recall, F1-Score, ROC-AUC, PR-AUC, Tasa de Falsos Positivos y los tiempos de entrenamiento y predicción.

5.2. Análisis de Métricas Clave para Detección de Fraude

5.2.1. Métricas Principales

Cuando trabajamos con detección de fraude, elegir y analizar correctamente las métricas es fundamental. Un Accuracy alto puede llevarnos a conclusiones erróneas en datasets desbalanceados. En nuestro caso, con solo un 0.17% de fraudes, un modelo que nunca detecte ningún fraude podría alcanzar un 99.83% de Accuracy simplemente clasificando todas las transacciones como legítimas.

Tabla 1: Métricas de rendimiento operacional por modelo y técnica de balanceo, marcando el **mejor valor** y el segundo mejor valor de cada métrica.

Método	Modelo	Accuracy	Precision	Recall	F1-Score	ROC-AUC	PR-AUC	False Positive Rate	Train Time (s)	Prediction Time (s)
Original	Random Forest	0.999561	<u>0.929412</u>	0.806122	0.863388	0.952769	0.872467	0.000106	58.13	0.25
	K-Nearest Neighbors	0.999508	0.960526	0.744898	0.839080	0.918246	0.860121	0.000053	0.04	53.25
	SVM Polinómico	0.999122	0.760870	0.714286	0.736842	0.947699	0.797347	0.000387	255.46	0.58
	SVM Sigmoide	0.998560	0.617647	0.428571	0.506024	0.731252	0.347016	0.000457	57.15	1.22
	SVM RBF	0.999280	0.782178	0.806122	0.793970	0.945351	0.787897	0.000387	111.33	3.21
Smote	Random Forest	0.999491	0.870968	0.826531	0.848168	0.973011	<u>0.866403</u>	0.000211	139.45	0.31
	K-Nearest Neighbors	0.997507	0.393204	0.826531	0.532895	0.922792	0.705311	0.002198	0.07	66.09
	SVM Polinómico	0.983866	0.087437	0.887755	0.159195	0.973981	0.678941	0.015968	14614.90	157.55
	SVM Sigmoide	0.736684	0.004983	0.765306	0.009902	0.819929	0.163776	<u>0.263365</u>	19347.60	389.55
	SVM RBF	0.984621	0.091387	0.887755	0.165714	0.966054	0.713080	0.015212	7507.45	228.15
Adasyn	Random Forest	0.999473	0.895349	0.785714	0.836957	0.973389	0.865395	0.000158	149.85	0.39
	K-Nearest Neighbors	0.997490	0.391304	0.826531	0.531148	0.922786	0.703043	0.002216	0.04	68.62
	SVM Polinómico	0.958112	0.035337	0.887755	0.067969	0.970109	0.632394	0.041766	21260.90	180.83
	SVM Sigmoide	0.590569	0.003379	0.806122	0.006729	0.723962	0.071531	0.409802	46297.06	1083.61
	SVM RBF	0.969840	0.047486	0.867347	0.090042	0.941043	0.613980	0.029984	15985.14	377.81
Smote Tomek	Random Forest	0.999491	0.870968	0.826531	0.848168	0.973011	<u>0.866403</u>	0.000211	139.64	0.72
	K-Nearest Neighbors	0.997507	0.393204	0.826531	0.532895	0.922792	0.705311	0.002198	0.07	51.04
	SVM Polinómico	0.983866	0.087437	0.887755	0.159195	0.973981	0.678941	0.015968	11442.42	102.95
	SVM Sigmoide	0.736684	0.004983	0.765306	0.009902	0.819929	0.163776	<u>0.263365</u>	18872.24	354.85
	SVM RBF	0.984621	0.091387	0.887755	0.165714	0.966054	0.713080	0.015212	6094.58	208.32

El Recall es la métrica más importante para nosotros, ya que mide qué proporción de fraudes reales conseguimos identificar. Por eso, nuestro objetivo principal es maximizar el Recall para identificar la mayor cantidad posible de fraudes reales.

La Precision tiene menor relevancia en nuestro caso. Es preferible generar algunas falsas alarmas que se pueden resolver con verificación manual, que perder fraudes reales que resultan en pérdidas económicas directas. Por esta razón, el F1-Score, que depende del equilibrio entre Precision y Recall, tampoco es nuestra métrica principal, ya que podría penalizar modelos con alto Recall pero Precision moderada.

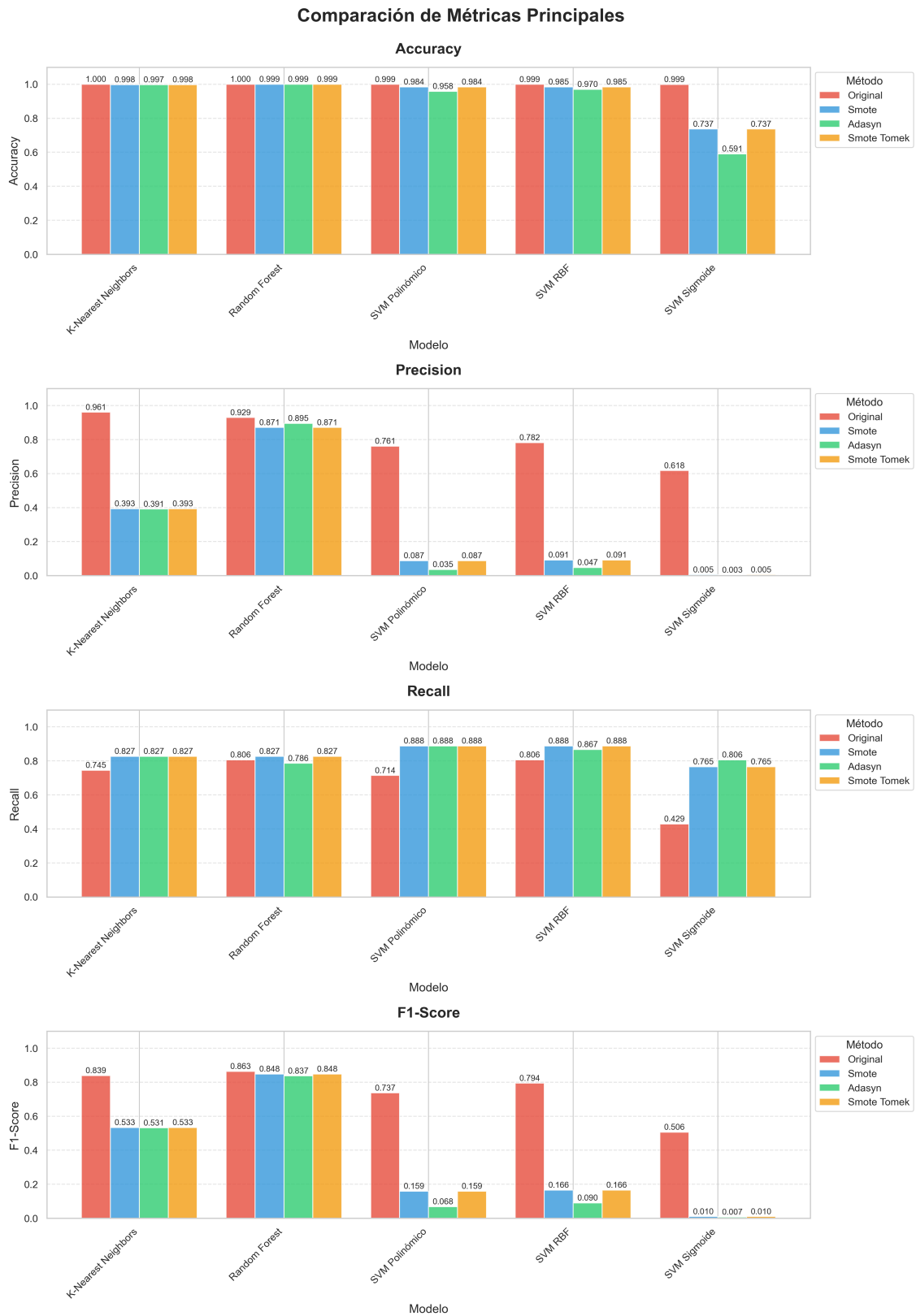


Figura 6: Comparación de métricas principales por modelo y técnica de balanceo. Se observa claramente cómo las técnicas de oversampling (SMOTE, ADASYN, SMOTE-Tomek) mejoran significativamente el Recall respecto al dataset original, mientras que el Accuracy se mantiene alto en todos los casos. Los SVM con kernels RBF y Polinómico alcanzan los mejores valores de Recall (0.888) con las técnicas de oversampling.

5.2.2. Curvas ROC y PR

Las curvas ROC y PR nos dan una visión más detallada de cómo funcionan los clasificadores. En nuestro contexto de detección de fraude, la curva ROC es la más relevante porque evalúa la capacidad general del modelo para distinguir entre transacciones legítimas y fraudulentas.

La curva ROC muestra la relación entre la capacidad del modelo para detectar fraudes (Recall) y su tendencia a generar falsas alarmas (Tasa de Falsos Positivos). Lo importante es que esta comparación nos da una visión equilibrada del rendimiento del modelo independientemente del desbalance en nuestros datos.

El AUC-ROC (Area Under Curve - Área Bajo la Curva) resume en un solo número qué tan bien puede discriminar el modelo entre las dos clases. El AUC mide el área que hay debajo de la curva ROC. Un AUC-ROC de 0.5 significa que el modelo no es mejor que adivinar al azar, mientras que un valor cercano a 1.0 indica que el modelo tiene una excelente capacidad de separación entre clases.

La curva PR compara Precision contra Recall, pero depende mucho de la Precision, que como ya establecimos no es nuestra métrica prioritaria en detección de fraude. Un modelo excelente para detectar fraudes puede parecer peor en la curva PR solo porque genera algunas alertas adicionales que necesitan verificación manual. Como preferimos verificar algunas transacciones extra antes que perder fraudes reales, nos enfocamos principalmente en las curvas ROC.

Las curvas ROC y Precision-Recall completas para todas las técnicas de balanceo se presentan en el Anexo (ver Figuras 11 a 16).

5.3. Matrices de Confusión

Las matrices de confusión nos muestran directamente los resultados de clasificación:

- **Verdaderos Positivos (TP):** fraudes que identificamos correctamente
- **Verdaderos Negativos (TN):** transacciones legítimas que clasificamos correctamente
- **Falsos Positivos (FP):** transacciones legítimas que marcamos incorrectamente como fraude

- **Falsos Negativos (FN):** fraudes que no detectamos

Un falso negativo ocurre cuando una transacción fraudulenta pasa desapercibida para nuestro modelo, y esto puede tener consecuencias económicas graves. Por ejemplo, si no detectamos un fraude de 10.000€, perdemos directamente esa cantidad. Un falso negativo representa una pérdida económica directa que normalmente ya no se puede recuperar. En detección de fraude, minimizar los falsos negativos es lo más importante, ya que a diferencia de los falsos positivos, que pueden solucionarse con verificación adicional, representan pérdidas irrecuperables.

Las matrices de confusión detalladas para todas las técnicas de balanceo y modelos evaluados se presentan en el Anexo (ver Figuras 7 a 10).

Capítulo 6

Discusión y Conclusiones

En esta sección analizamos los resultados presentados en la Sección anterior. Interpretamos las métricas que obtuvimos y sacamos conclusiones importantes sobre cómo funcionan los modelos y qué tan efectivas son las estrategias de balanceo de datos.

6.1. Análisis Comparativo de Estrategias de Balanceo

6.1.1. Dataset Original: Las Limitaciones del Desbalance Extremo

Cuando analizamos los resultados del dataset original, vemos una tendencia clara que confirma los problemas del desbalance extremo. Aunque los modelos alcanzan un Accuracy muy alto (superior al 99.9%), esta métrica resulta engañosa debido al desbalance, de calificar todos los datos como no fraudulentos, se obtendría este resultado.

Una métrica que refleja el objetivo del problema, es el Recall promedio. Esta métrica evalúa cuantos casos fraudulentos han sido detectados. Los modelos entrenados con el dataset original obtienen un recall de apenas 0.70, con un rango que va desde 0.43 hasta 0.81. Esto significa que, en promedio, los modelos solo detectan 7 de cada 10 fraudes, dejando pasar aproximadamente el 30 % de los casos fraudulentos.

Otra métrica relevante, es el ROC-AUC ya que nos refleja no sólo esos casos fraudulentos que se han detectado, si no también los falsos fraudulentos predichos por el algoritmo. El promedio obtenido entrenando con el dataset original es de 0.899, que

aunque parece aceptable, varía considerablemente entre modelos (desde 0.73 hasta 0.95). Esta variabilidad indica que algunos algoritmos tienen más dificultades que otros para distinguir entre clases cuando hay un desbalance tan extremo.

6.1.2. Métodos de Oversampling

La aplicación de técnicas de oversampling (SMOTE, ADASYN y SMOTE-Tomek) cambia completamente el rendimiento de detección de fraudes. Los resultados muestran una mejora consistente en las métricas importantes.

Todos los métodos de oversampling elevan el Recall promedio a aproximadamente 0.84, lo que representa una mejora del 20 % respecto a la evaluación sobre el dataset original. Más importante aún, el rango de Recall se vuelve más estrecho y más alto (0.77-0.89), indicando que incluso los modelos con peor rendimiento logran detectar más del 75 % de los fraudes.

El ROC-AUC promedio aumenta a más de 0.93 para SMOTE y SMOTE-Tomek, y 0.91 para ADASYN. Esta mejora demuestra que los modelos no solo detectan más fraudes, sino que también mejoran su capacidad general de discriminación entre clases.

Es notable que SMOTE y SMOTE-Tomek producen resultados prácticamente idénticos, mientras que ADASYN muestra un rendimiento ligeramente inferior pero aún muy superior al dataset original.

6.2. Rendimiento de los Modelos de Clasificación

6.2.1. Random Forest

Random Forest demuestra ser el modelo más estable y equilibrado a través de todas las estrategias de balanceo. Con las técnicas de oversampling, alcanza un Recall excelente de 0.827 y mantiene un ROC-AUC superior a 0.973, demostrando tanto alta capacidad de detección de fraudes como excelente discriminación general entre clases.

Aunque no son nuestras métricas principales, Random Forest también destaca en otras medidas de rendimiento: logra una Precision de 0.871, y un F1-Score de 0.848,

indicando un balance muy sólido entre todas las métricas. Su tasa de falsos positivos es muy baja (0.0002), generando pocas alertas innecesarias.

Además, sus tiempos de entrenamiento son razonables (58-139 segundos) y los tiempos de predicción muy rápidos (0.25-0.73 segundos), lo que lo convierte en la opción más viable para entornos de producción que requieren respuestas rápidas.

6.2.2. K-Nearest Neighbors

K-NN muestra una de las mejores transformaciones con oversampling: mejora de un Recall de 0.745 en el dataset original a 0.827 con todas las técnicas de oversampling. Su ROC-AUC también mejora, pasando de 0.918 a más de 0.92.

La gran ventaja de K-NN es su velocidad de entrenamiento extremadamente rápida (0.04-0.07 segundos), lo que permite experimentación rápida y reentrenamiento frecuente. Sus tiempos de predicción (51-68 segundos) son rápidos para el volumen de datos que maneja, haciendo viable su uso en sistemas de producción donde se procesan transacciones por lotes.

6.2.3. Support Vector Machines

Los SVM muestran el comportamiento más notable con oversampling. Los kernels RBF y Polinómico alcanzan los mejores resultados absolutos: Recall de 0.888, el más alto de todos los modelos, y ROC-AUC muy altos (0.966-0.974), superando a todos los demás modelos en capacidad pura de detección de fraudes. Estos resultados demuestran que los SVM pueden extraer el máximo provecho de los datos sintéticos generados por las técnicas de oversampling.

Sin embargo, este rendimiento superior viene con tiempos de entrenamiento considerablemente lentos: especialmente para el kernel Polinómico (hasta 6 horas), lo que puede ser problemático para reentrenamientos frecuentes. No obstante, sus tiempos de predicción (102-377 segundos) son rápidos considerando la complejidad del modelo y el volumen de datos procesados.

El SVM Sigmoide presenta generalmente peores resultados en todas las técnicas de oversampling comparando con otros modelos, con Recall de 0.765 y ROC-AUC de 0.820, además de mantener los tiempos de entrenamiento lentos característicos de los SVM.

Los kernels RBF y Polinómico son la mejor opción cuando la máxima detección de fraudes es prioritaria y se dispone de recursos computacionales abundantes.

6.3. Comparación con Trabajos Previos: Competitividad con Eficiencia

Para poner nuestros resultados en perspectiva, los comparamos con el trabajo de referencia "Deep Anomaly Detection with Deviation Networks (DevNet)" que reporta para el mismo dataset:

- ROC-AUC: 0.980
- AUC-PR: 0.690

Nuestros mejores resultados con Random Forest y oversampling alcanzan:

- ROC-AUC: 0.973
- PR-AUC: 0.866

Hemos conseguido resultados competitivos utilizando modelos tradicionales de aprendizaje automático. Aunque nuestro ROC-AUC es ligeramente inferior al de DevNet, nuestro PR-AUC es considerablemente superior, y nuestro Recall supera las expectativas para este tipo de problemas.

Otra diferencia significativa está en la eficiencia computacional. Mientras que las redes neuronales requieren recursos GPU especializados, largos tiempos de entrenamiento y una optimización compleja de hiperparámetros; nuestros modelos tradicionales se caracterizan por tiempos de entrenamiento y predicción predecibles, un menor requerimiento de recursos hardware al funcionar eficientemente con CPU en lugar de GPU, una mayor interpretabilidad de los resultados y una mayor facilidad tanto en su implementación como en su mantenimiento.

Bibliografía

- [1] A. Dal Pozzolo, O. Caelen, R. A. Johnson, and G. Bontempi, “Calibrating probability with undersampling for unbalanced classification,” in *IEEE Symposium on Computational Intelligence and Data Mining (CIDM)*, 2015, pp. 159–166.
- [2] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [3] A. C. Bahnsen, D. Aouada, A. Stojanovic, and B. Ottersten, “Feature engineering strategies for credit card fraud detection,” *Expert Systems with Applications*, vol. 51, pp. 134–142, 2016.
- [4] C. Whitrow, D. J. Hand, P. Juszczak, D. Weston, and N. M. Adams, “Transaction aggregation as a strategy for credit card fraud detection,” *Data Mining and Knowledge Discovery*, vol. 18, no. 1, pp. 30–55, 2009.
- [5] J. Jurgovsky, M. Granitzer, K. Ziegler, S. Calabretto, P.-E. Portier, L. He-Guelton, and O. Caelen, “Sequence classification for credit-card fraud detection,” *Expert Systems with Applications*, vol. 100, pp. 234–245, 2018.
- [6] H. He, Y. Bai, E. A. Garcia, and S. Li, “ADASYN: Adaptive synthetic sampling approach for imbalanced learning,” in *IEEE International Joint Conference on Neural Networks (IEEE World Congress on Computational Intelligence)*, 2008, pp. 1322–1328.
- [7] X.-Y. Liu, J. Wu, and Z.-H. Zhou, “Exploratory undersampling for class-imbalance learning,” *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 39, no. 2, pp. 539–550, 2009.
- [8] G. Pang, C. Shen, and A. van den Hengel, “Deep anomaly detection with deviation networks,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 353–362.
- [9] T. Sarkar, “XBNet: An extremely boosted neural network,” *Intelligent Systems with Applications*, vol. 12, p. 200052, 2021.

- [10] E. L. Paula, M. Ladeira, R. N. Carvalho, and T. Marzagão, “Deep learning anomaly detection as support fraud investigation in Brazilian exports and anti-money laundering,” in *15th IEEE International Conference on Machine Learning and Applications (ICMLA)*, 2016, pp. 954–960.
- [11] F. T. Liu, K. M. Ting, and Z.-H. Zhou, “Isolation forest,” in *Eighth IEEE International Conference on Data Mining*, 2008, pp. 413–422.
- [12] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [13] S. Ruder, “An overview of gradient descent optimization algorithms,” *arXiv preprint arXiv:1609.04747*, 2016.
- [14] The Nilson Report, “Payment card fraud losses reach \$28.65 billion,” Issue 1187, April 2021.
- [15] Machine Learning Group - ULB, “Credit Card Fraud Detection,” Kaggle Dataset, 2018. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>
- [16] Anthropic, “Claude” <https://www.anthropic.com/claude>
- [17] G. E. A. P. A. Batista, R. C. Prati, and M. C. Monard, “A study of the behavior of several methods for balancing machine learning training sets,” *ACM SIGKDD Explorations Newsletter*, vol. 6, no. 1, pp. 20–29, 2004.
- [18] I. Tomek, “Two modifications of CNN,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 6, no. 11, pp. 769–772, 1976.

Anexo

6.4. Matrices de Confusión

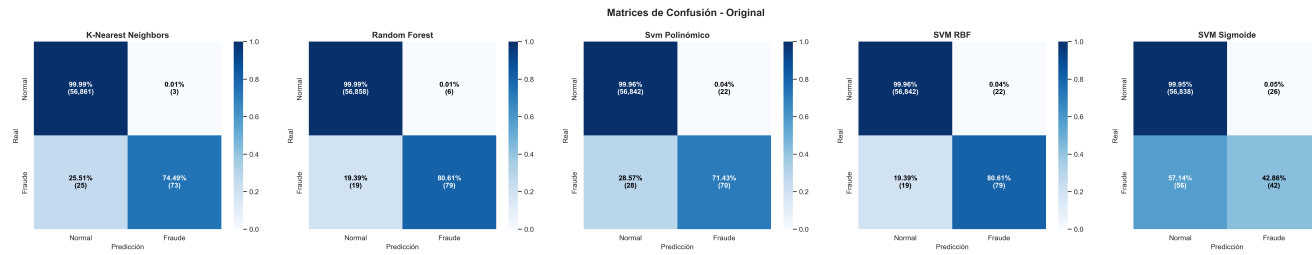


Figura 7: Matrices de confusión para todos los modelos entrenados con el dataset original. Se observa que todos los modelos logran identificar la mayoría de transacciones legítimas correctamente (valores altos en la diagonal superior izquierda), pero varían significativamente en su capacidad para detectar fraudes. Random Forest y SVM RBF muestran el mejor balance entre detección de fraudes y minimización de falsos positivos.

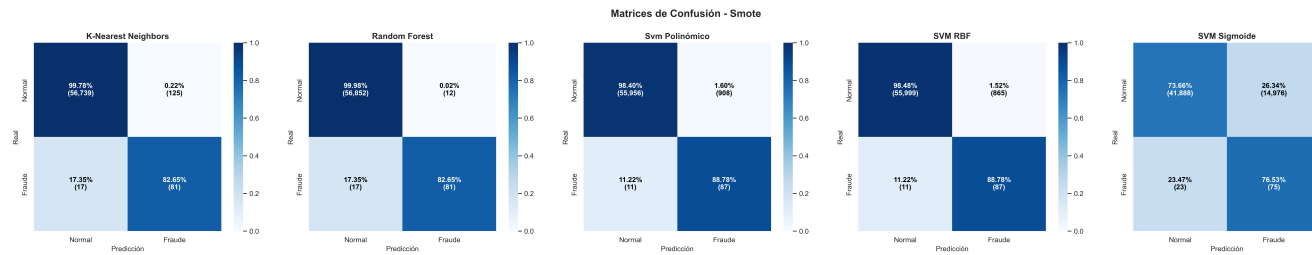


Figura 8: Matrices de confusión para todos los modelos entrenados con SMOTE. Los modelos SVM con kernels RBF y Polinómico logran detectar la mayoría de fraudes (valores altos en la diagonal inferior derecha), aunque a costa de generar más falsos positivos. Random Forest mantiene un mejor equilibrio entre detección de fraudes y control de falsas alarmas.

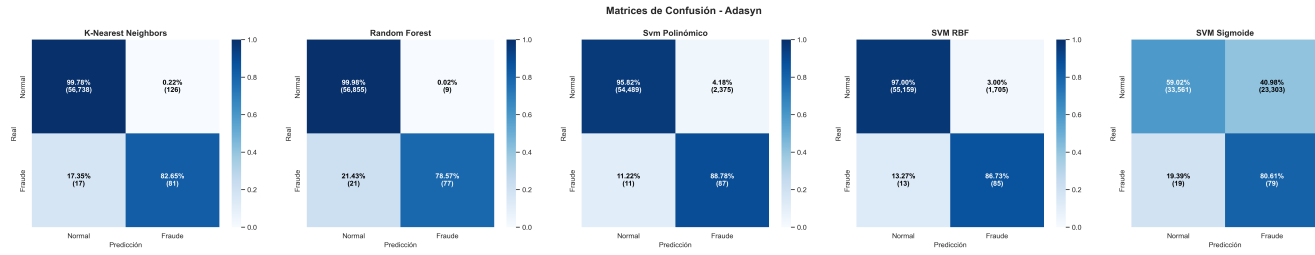


Figura 9: Matrices de confusión para todos los modelos entrenados con ADASYN. Los resultados son similares a SMOTE, con SVM RBF y Polinómico mostrando alta capacidad de detección de fraudes pero mayor tasa de falsos positivos. K-NN y Random Forest presentan un comportamiento más conservador con menos falsas alarmas.

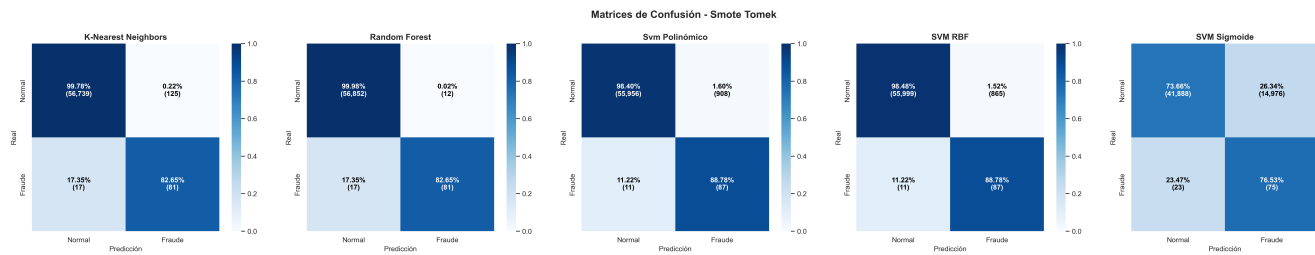


Figura 10: Matrices de confusión para todos los modelos entrenados con SMOTE-Tomek. Los resultados son prácticamente idénticos a SMOTE, confirmando que la limpieza adicional de casos ambiguos mediante Tomek Links no afecta significativamente el rendimiento de los modelos en este dataset.

6.5. Curvas ROC

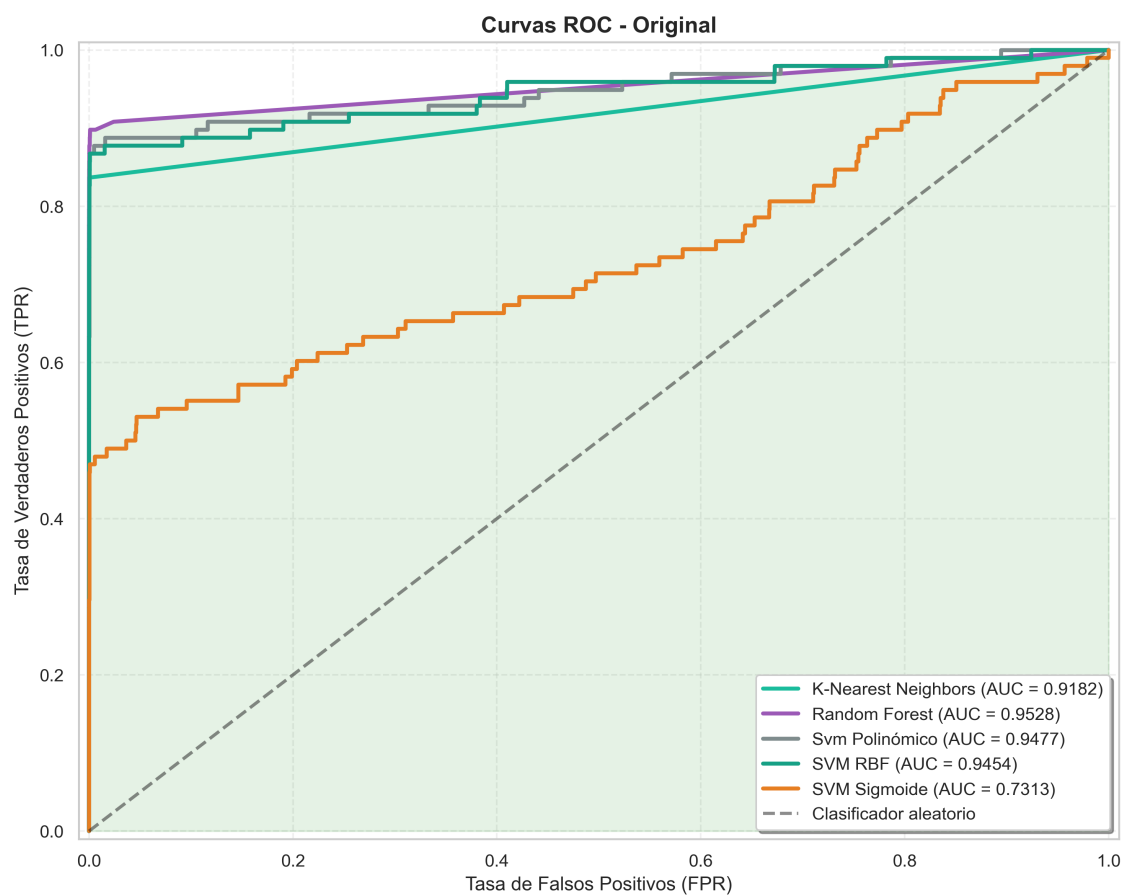


Figura 11: Curvas ROC para modelos entrenados con el dataset original. Random Forest presenta el mejor AUC-ROC (0.953), seguido por SVM Polinómico (0.948) y SVM RBF (0.945). Todas las curvas se sitúan considerablemente por encima de la línea diagonal (clasificador aleatorio), indicando capacidad discriminativa significativa incluso sin técnicas de balanceo.

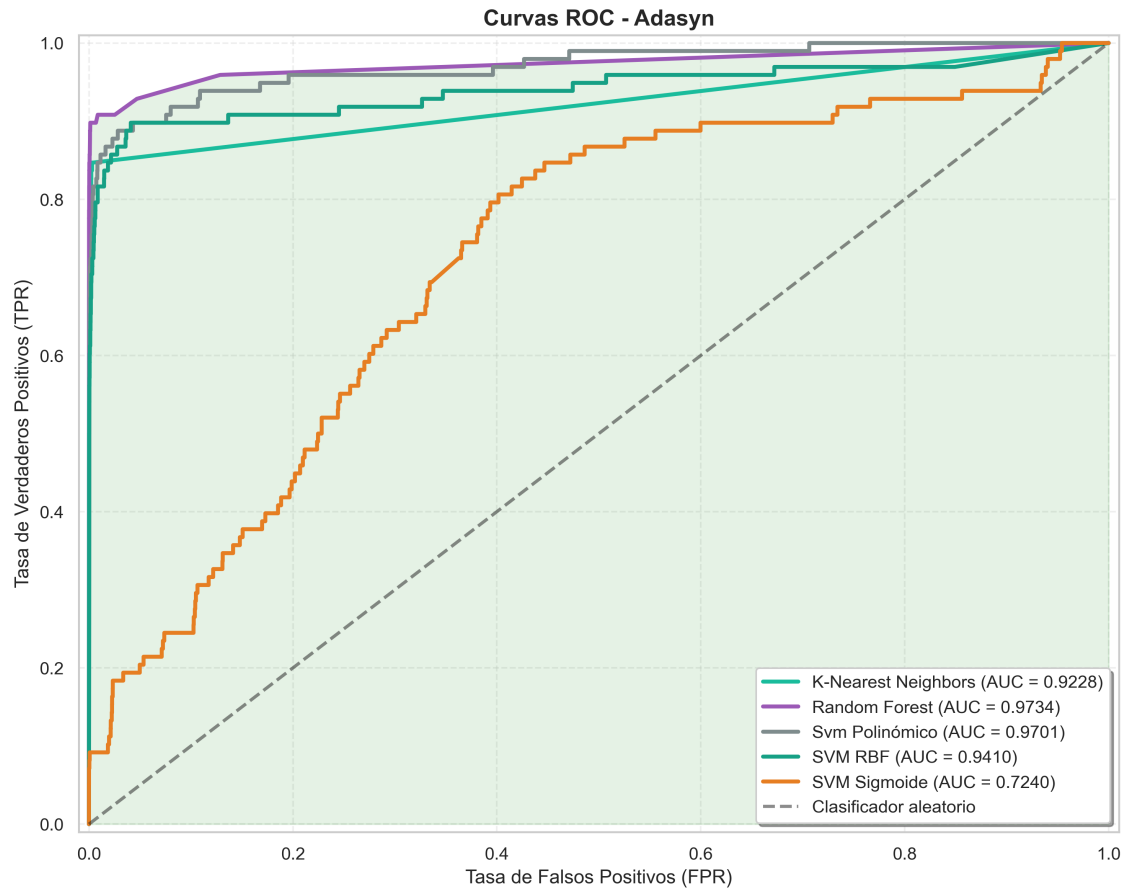


Figura 12: Curvas ROC para modelos entrenados con ADASYN. Los SVM con kernels RBF (0.941) y Polinómico (0.970) junto con Random Forest (0.973) muestran excelente capacidad discriminativa. Las técnicas de oversampling mejoran el rendimiento general, especialmente en la región de alta sensibilidad (esquina superior izquierda).

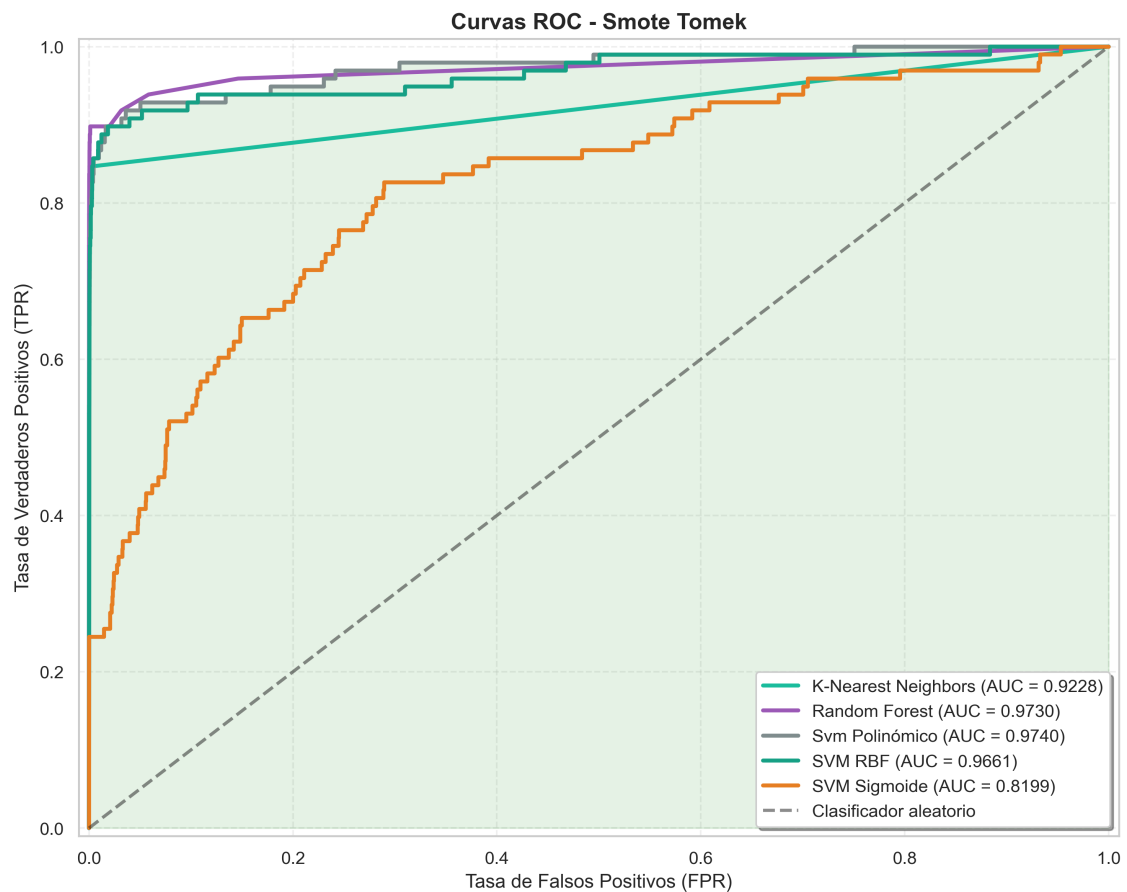


Figura 13: Curvas ROC para modelos entrenados con SMOTE-Tomek. Los resultados son idénticos a SMOTE, con SVM Polinómico (0.974) y Random Forest (0.973) alcanzando los mejores AUC-ROC. La consistencia entre SMOTE y SMOTE-Tomek confirma la robustez de estos algoritmos ante variaciones en las técnicas de balanceo.

6.6. Curvas Precision-Recall

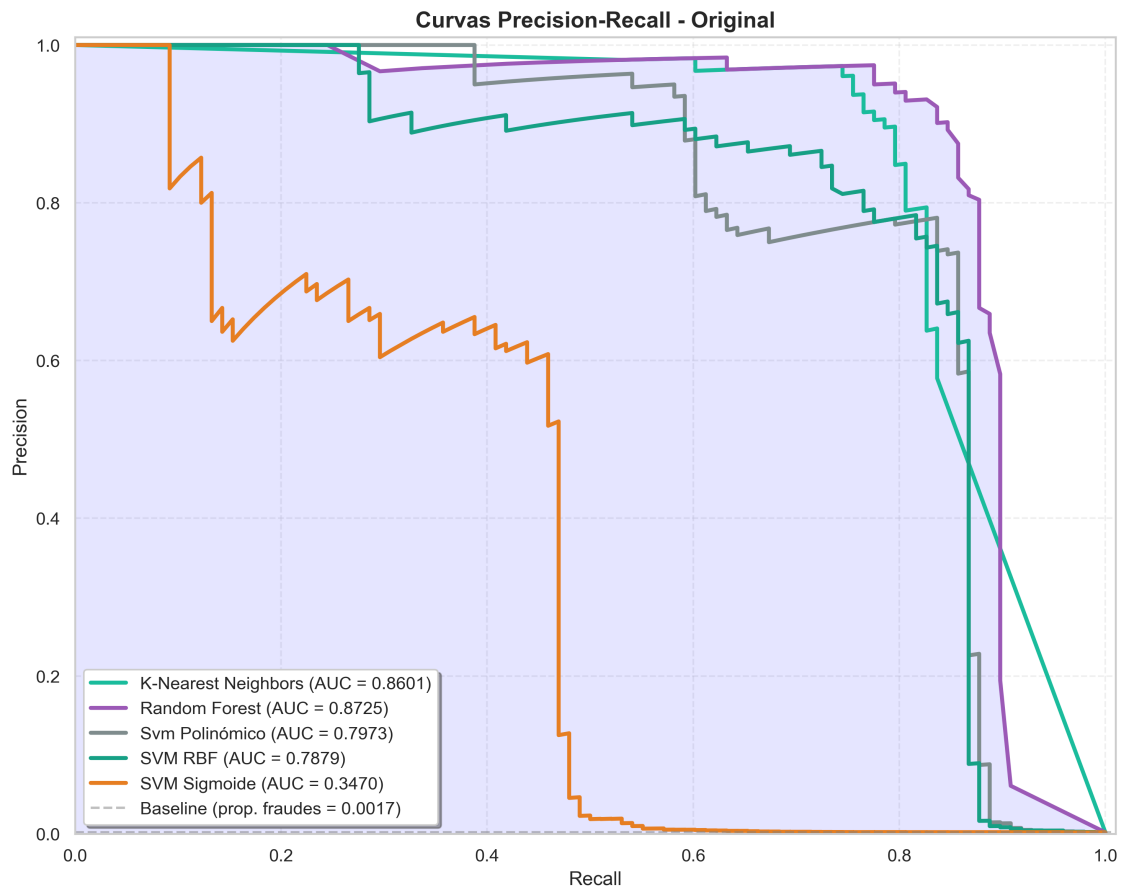


Figura 14: Curvas Precision-Recall para modelos entrenados con el dataset original. Random Forest presenta el mejor AUC-PR (0.873), seguido por K-NN (0.860). Las curvas muestran que mantener alta precisión se vuelve cada vez más difícil a medida que aumentamos el recall, especialmente para SVM Sigmoide que muestra el peor rendimiento con caídas abruptas en precisión.

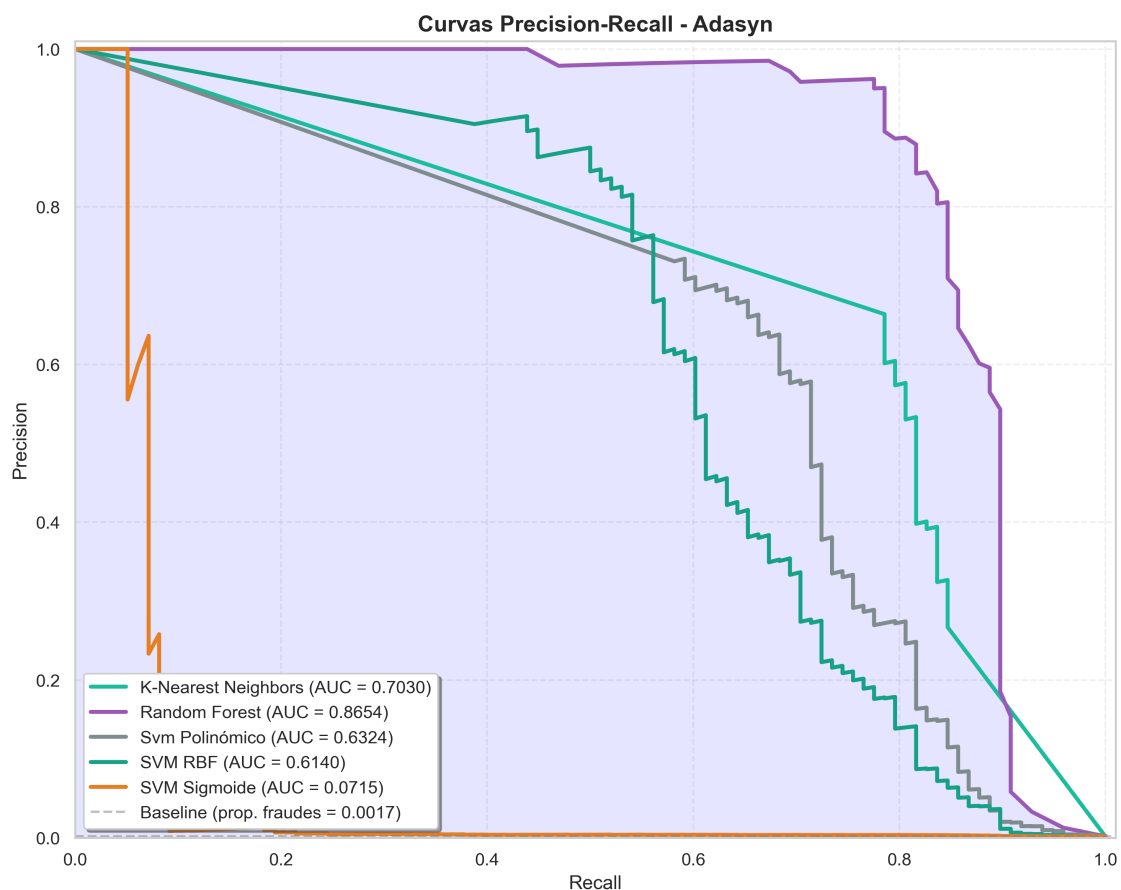


Figura 15: Curvas Precision-Recall para modelos entrenados con ADASYN. Random Forest mantiene el mejor AUC-PR (0.865), mientras que SVM Sigmoide continúa mostrando el peor rendimiento (0.072). Las técnicas de oversampling mejoran el comportamiento general pero SVM Sigmoide sigue siendo problemático para este dataset específico.

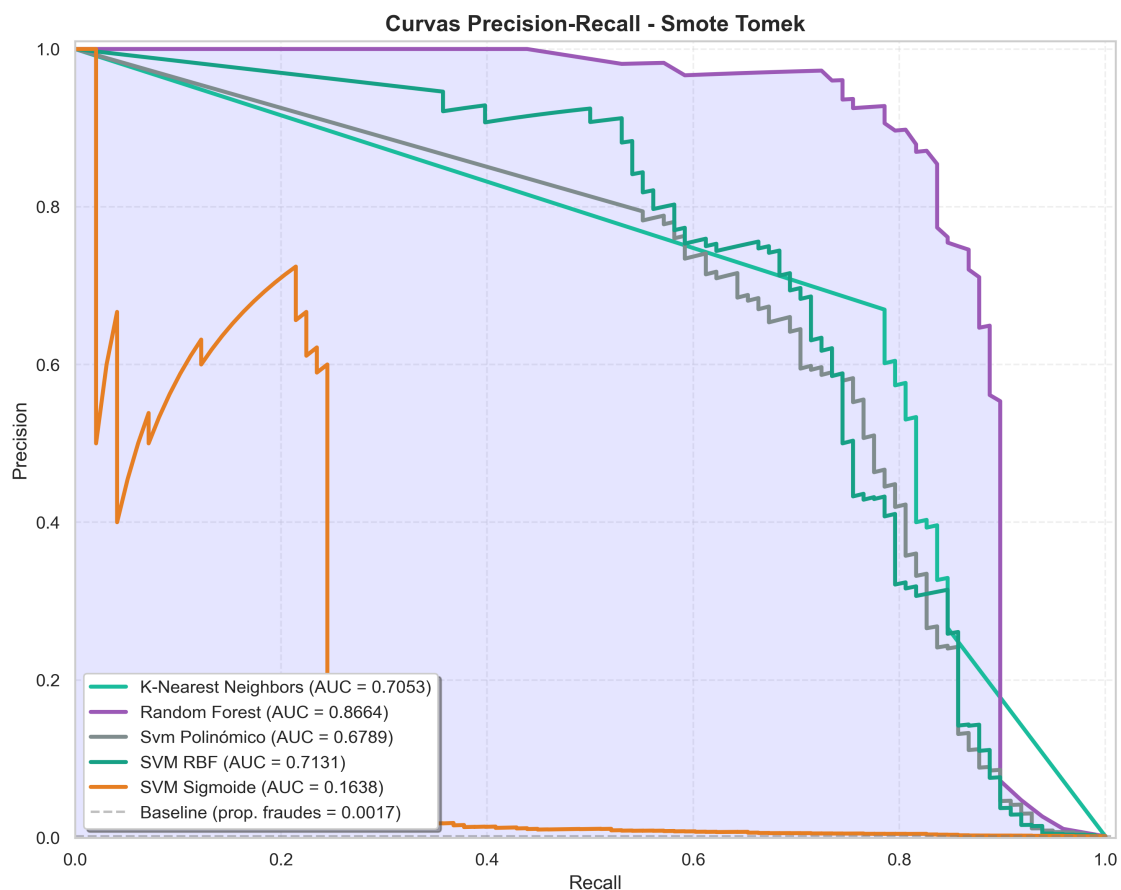


Figura 16: Curvas Precision-Recall para modelos entrenados con SMOTE-Tomek. Los resultados replican exactamente los obtenidos con SMOTE, con Random Forest alcanzando el mejor AUC-PR (0.866) y manteniendo una curva suave que indica estabilidad en el balance precision-recall a través de diferentes umbrales de clasificación.